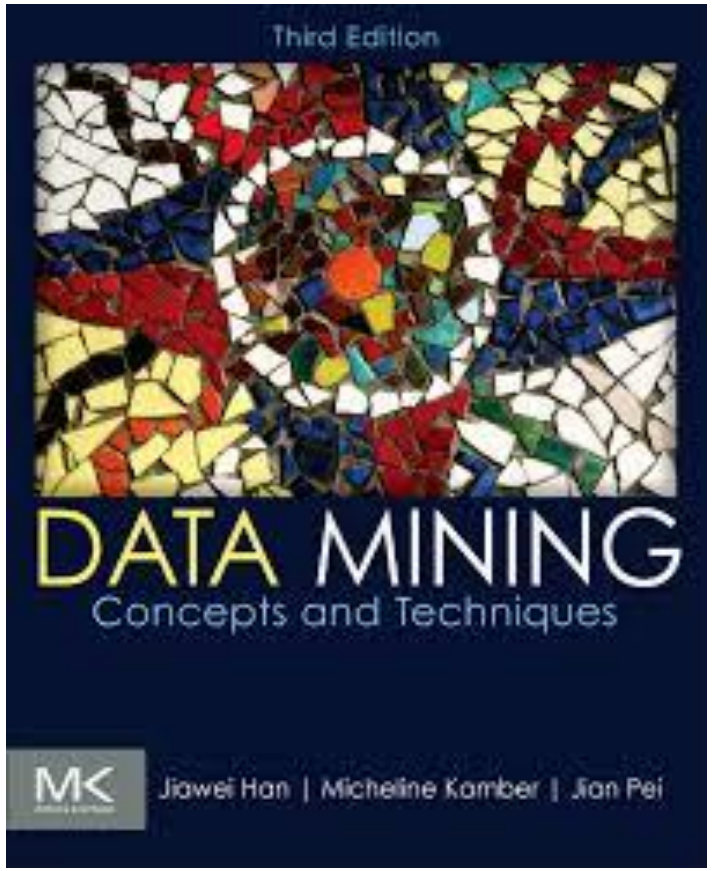
Machine Learning and Data Mining Basics

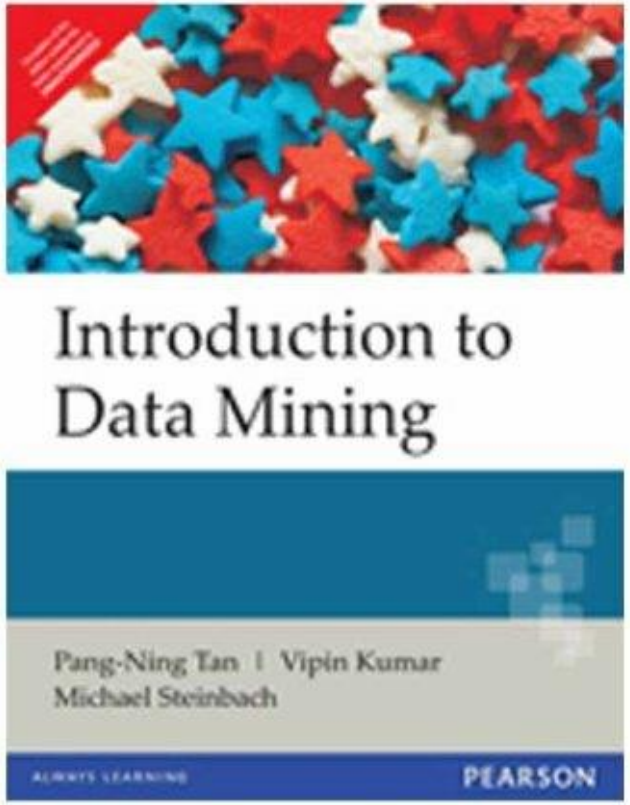
Course Goal

- **Differentiate Data Mining and Machine Learning** – Understand the scope, purpose, and key differences.
- **Understand Data Representation** – Learn how data is structured, formatted, and encoded for ML.
- **Master Data Preprocessing** – Handle missing data, scaling, normalization, and feature engineering.
- **Comprehend the Machine Learning Life Cycle** – From problem definition to model deployment.
- **Identify Types of Machine Learning** – Supervised, Semi-supervised, Unsupervised, and Reinforcement Learning.
- **Implement Core ML Algorithms** – Linear & logistic regression, decision trees, SVM, KNN, Naïve Bayes, etc.
- **Evaluate ML Models** – Metrics, validation, cross-validation, and performance improvement techniques.
- **Introduction to Deep Learning** – Basics of neural networks and their applications.
- **Develop Practical Thinking Skills** – Learn frameworks for problem-solving and common beginner pitfalls.

Course Materials and References

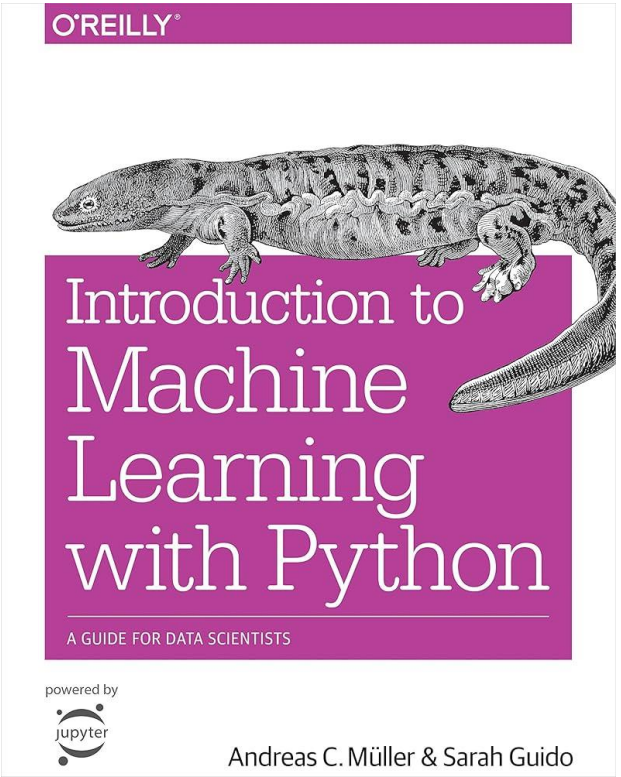


Data Mining Concepts and Techniques by Jiawei Han, Micheline Kamber, Jian Pei ·

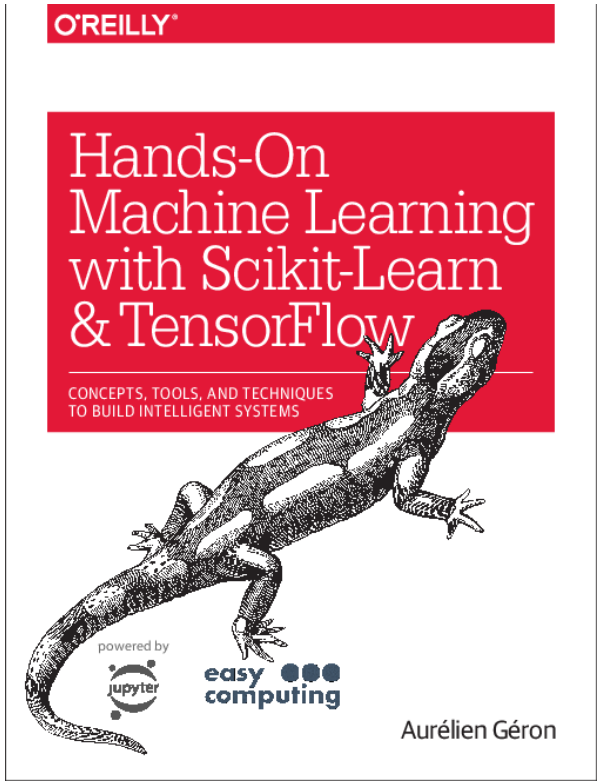


Introduction to Data Mining by Pang-Ning Tan Michael Steinbach Vipin Kumar

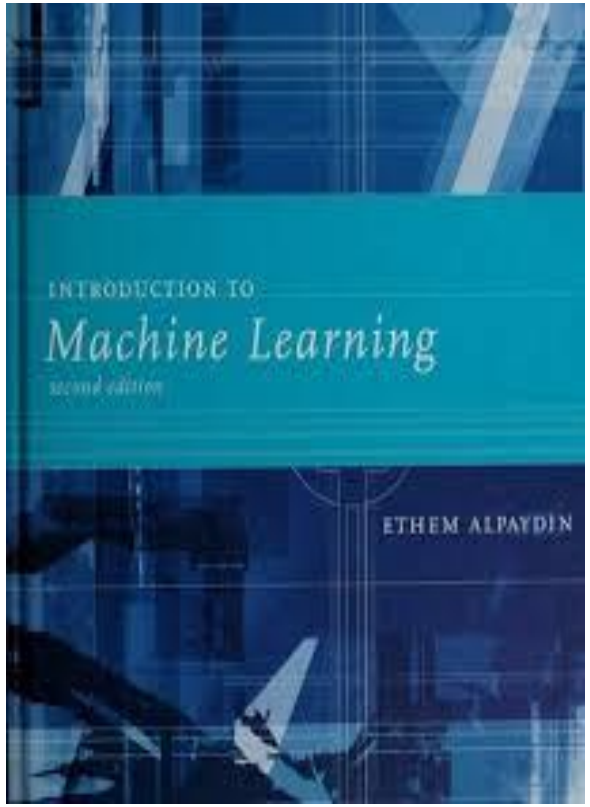
Course Materials and References



Introduction to Machine Learning with Python by Andreas C. Muller and Sarah Guido



Hands-On Machine Learning with Scikit-Learn and TensorFlow by Aurélien Géron



Introduction to Machine Learning by Ethem Alpaydın

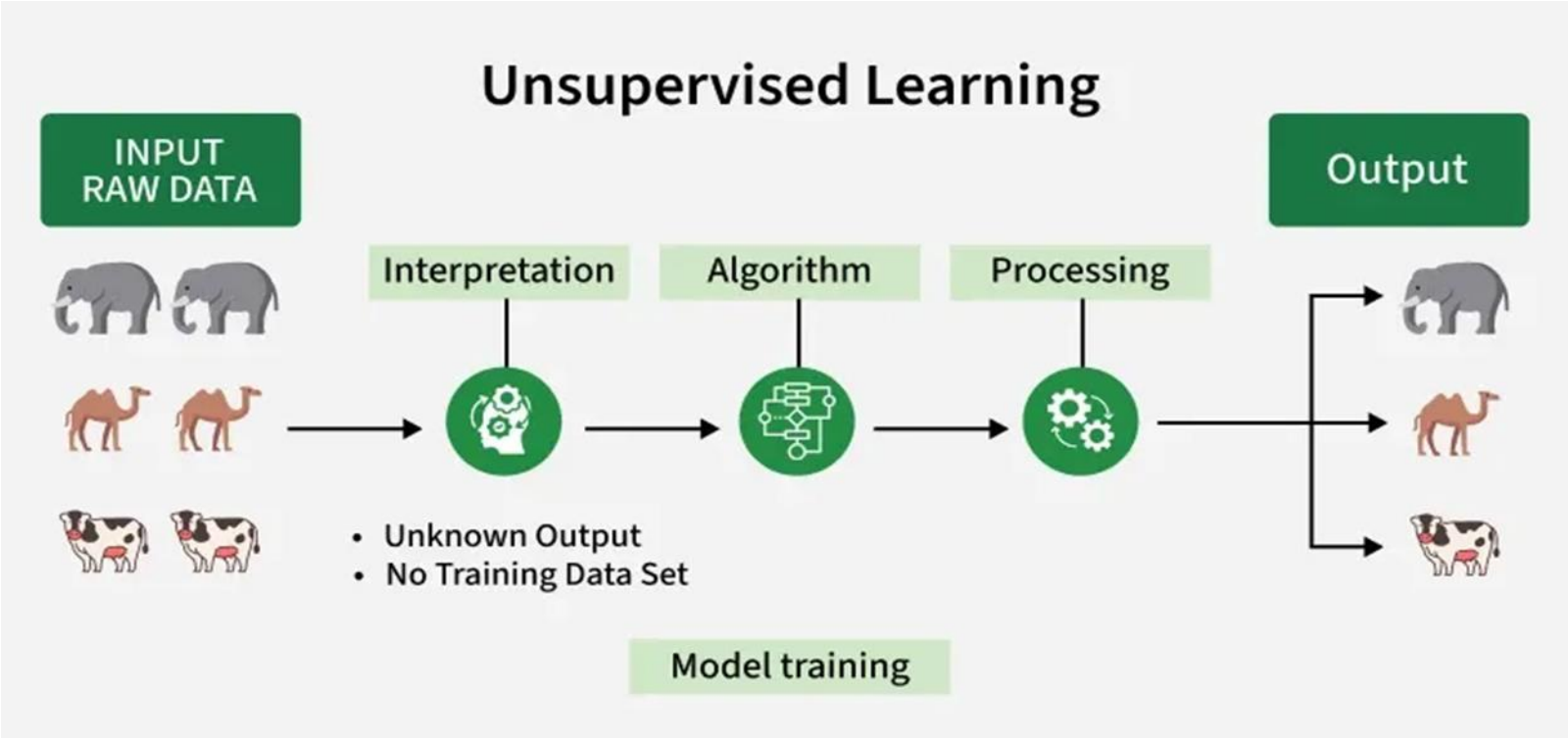
Outline

- Unsupervised Learning
- Clustering
- Applications of Clustering
- Types Of Clustering
- K-Means Algorithm
- How does k-means clustering work
- Convergence
- Elbow Method

- Neural Networks

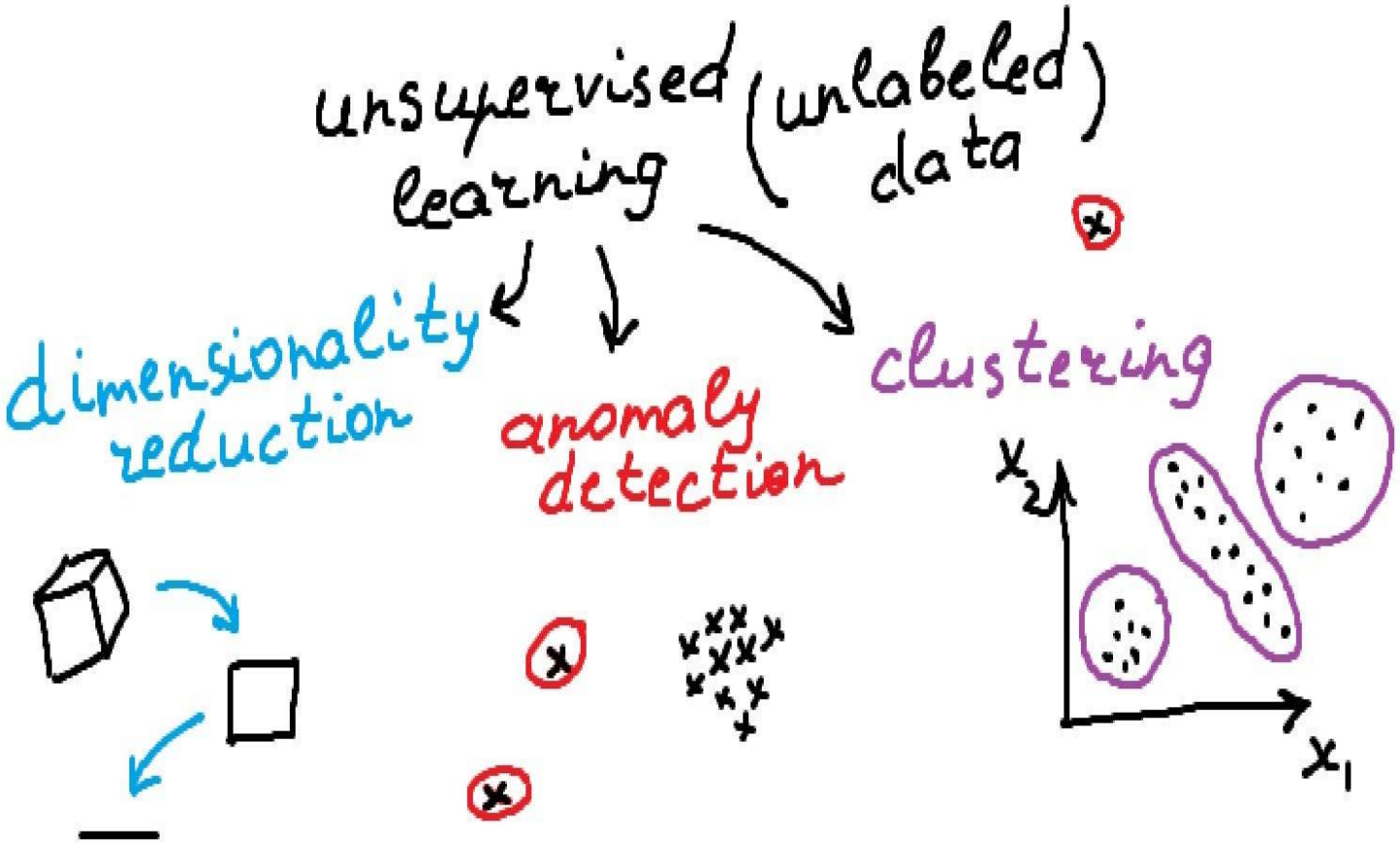
Unsupervised Learning

- Model works without labeled data (no prior knowledge of the data's meaning)
- Learns patterns on its own by grouping similar data points or finding hidden structures.
- No need for any human intervention.



Unsupervised Learning

- Used for tasks like:
  - Clustering
  - Dimensionality reduction
  - Anomaly Detection
  - Association Rule Learning



Clustering

- *Clustering* is the process of dividing a set of data objects(or observations) into subsets.
- Clustering is also called **data segmentation**
- Clustering is a form of learning by **observation**, rather than learning by **example.**

Clustering

- Each subset is a cluster, such that objects in a cluster are **similar** to one another, yet **dissimilar** to objects in other clusters.

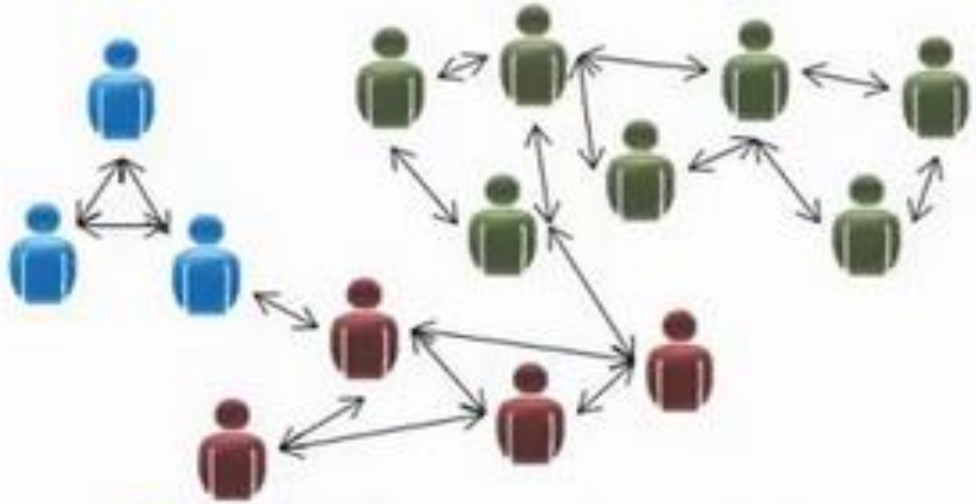
- Maximize the inter cluster similarity
- Minimize the intra cluster similarity

Intra cluster distance

Inter cluster distance

Applications of clustering

→ Market segmentation



→ Social network analysis

Organize computing clusters

Astronomical data analysis

Types of Clustering

Clustering

Partitioning Methods

Split data into K distinct clusters (hard or soft).

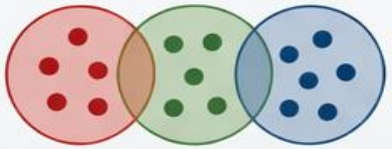
Hard (Crisp) Partitioning

- Each point belongs to exactly one cluster.

Example: **K-Means**

Soft (Fuzzy) Partitioning

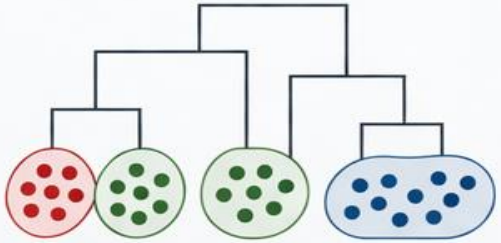
- Each point has membership in multiple clusters.



Example: **Fuzzy C-Means**

Hierarchical Methods

Build a hierarchy (dendrogram) of nested clusters.

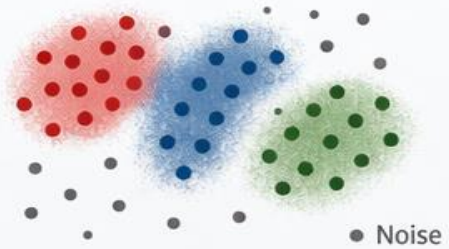


• Agglomerative (bottom-up) • Divisive (top-down)

Example: **Agglomerative Clustering**

Density-Based Methods

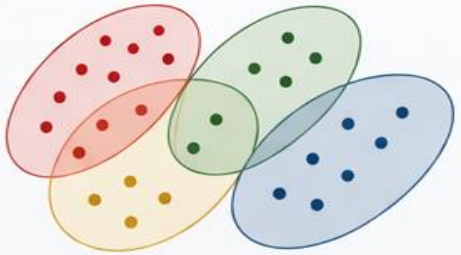
Find dense regions separated by sparse areas. Good for irregular shapes and noise.



Examples: **DBSCAN, OPTICS**

Model-Based Methods

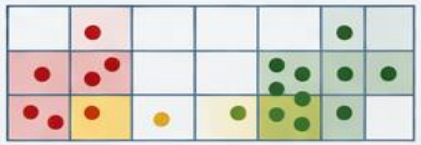
Assume data is generated from a statistical model and find the best fit.



Examples: **Gaussian Mixture Models (GMM), EM**

Grid-Based Methods

Divide the data space into a finite grid of cells and cluster based on cell density. Fast for large datasets.



Examples: **STING, CLIQUE**

Graph-Based Methods

Treat data as a graph and find groups of strongly connected nodes.

Examples: **Spectral Clustering, Community Detection**

Constraint-Based / Semi-Supervised

Incorporate prior knowledge or constraints (must-link / cannot-link) into clustering.

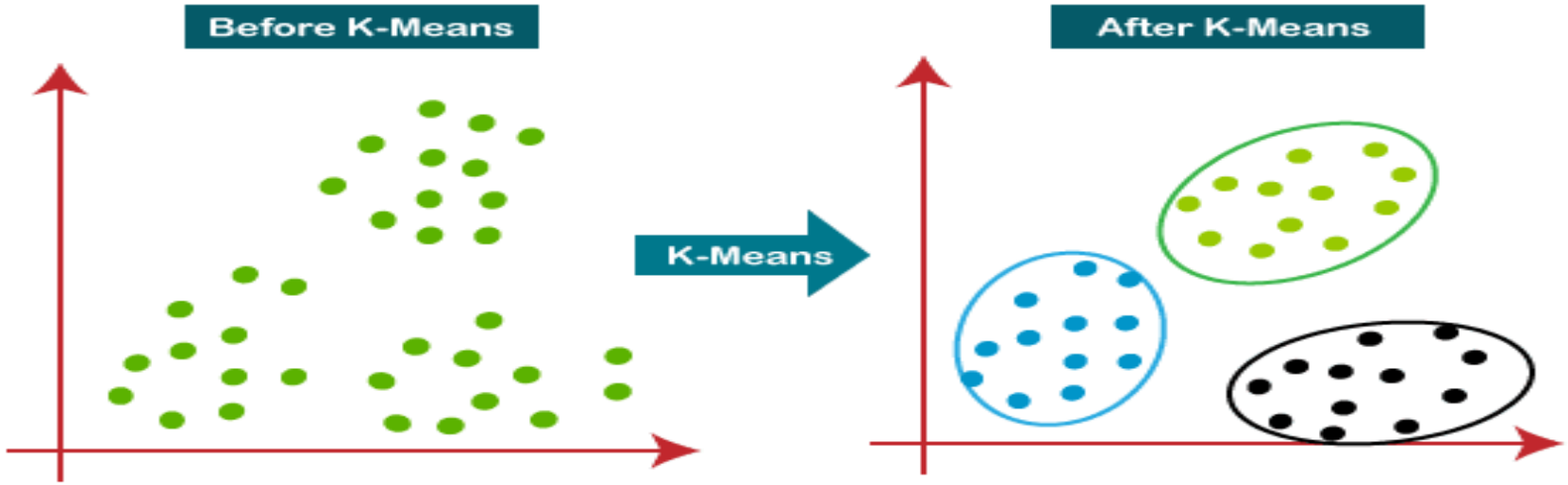
Examples: **COP-KMeans, Constrained Hierarchical**

K-Means Algorithm

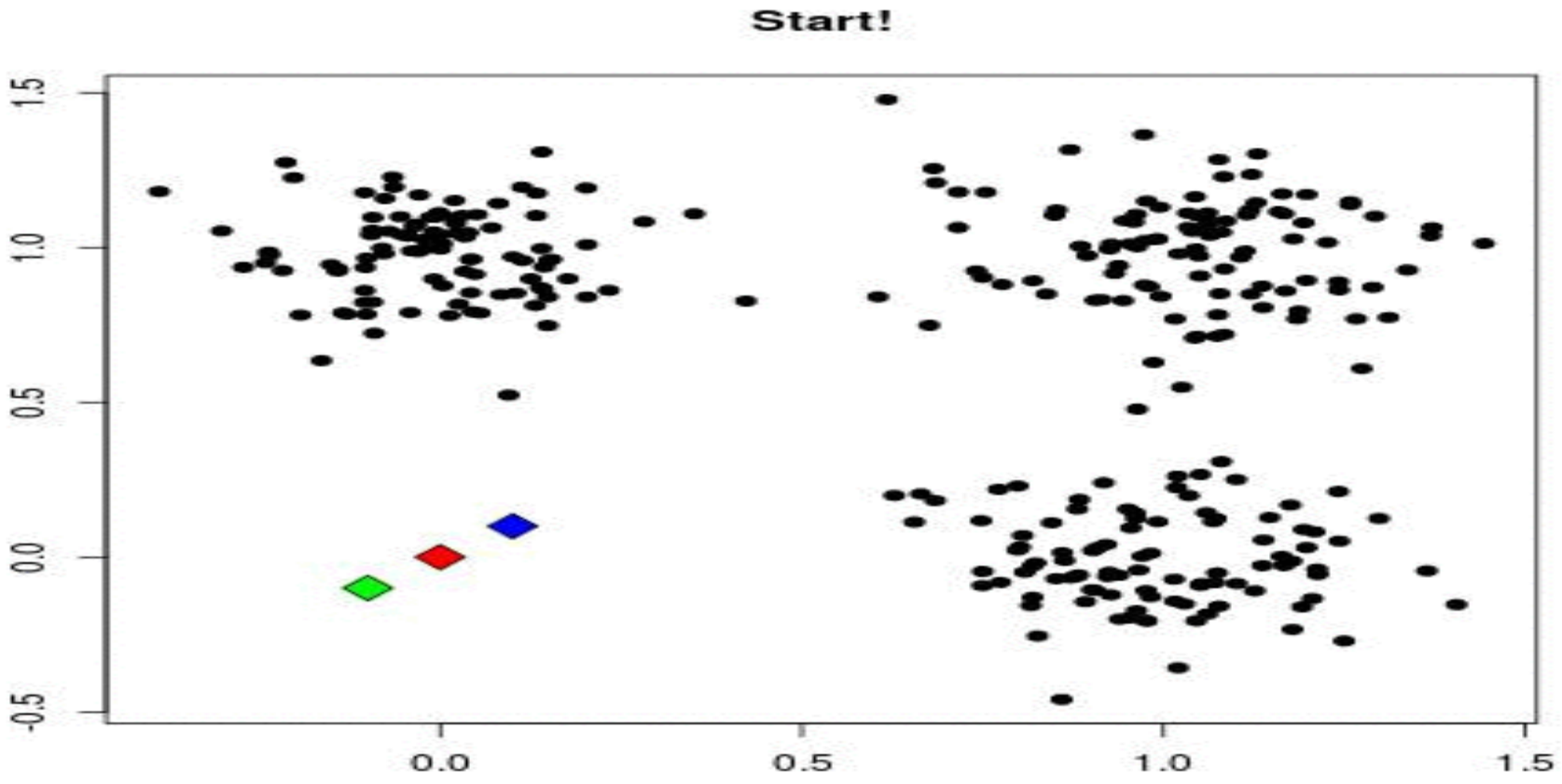
- Stipulates that a data point can exist in just one cluster (exclusive or "hard" clustering).

- An iterative, centroid-based clustering algorithm that partitions a dataset into similar groups based on the **distance** between their centroids (cluster center).

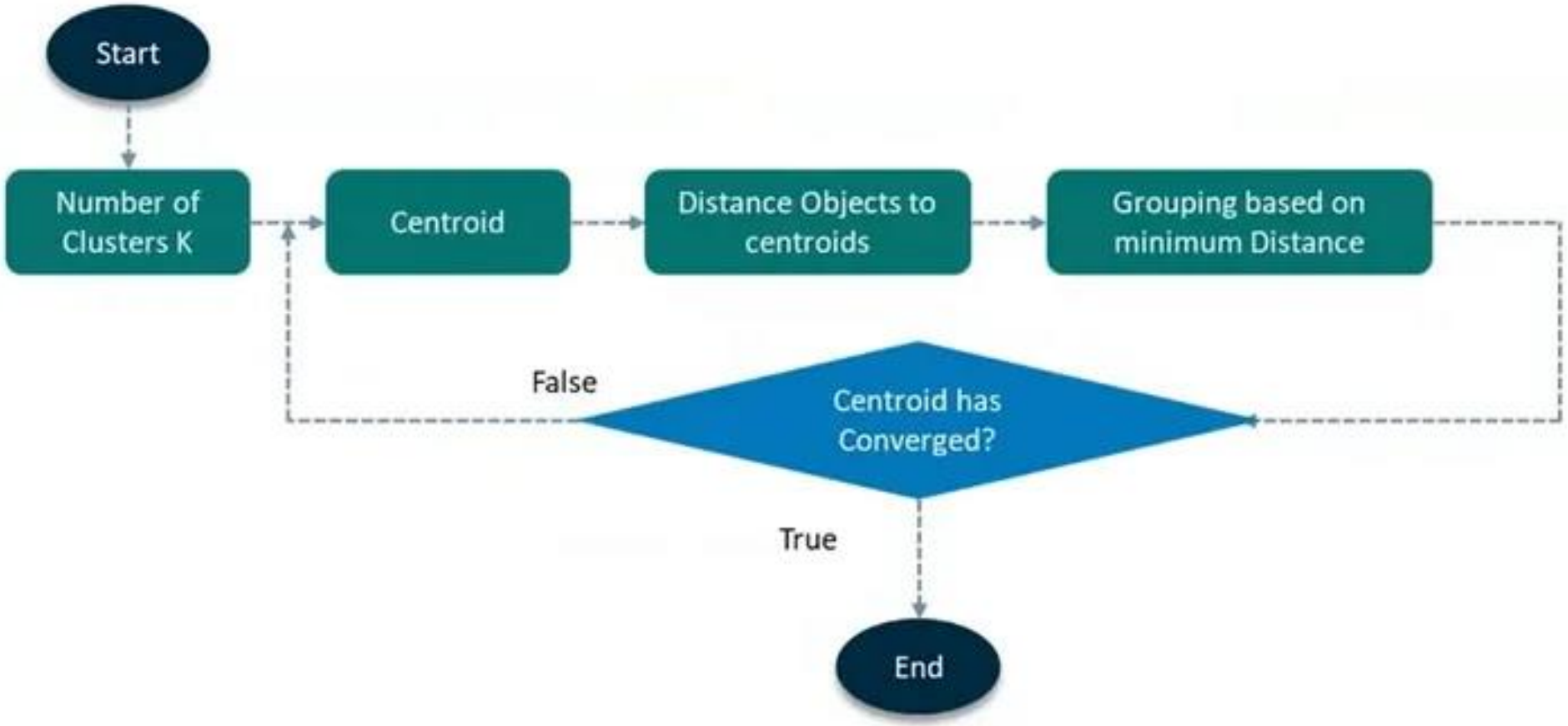
- The centroid, or cluster center, is either the **mean or median** of all the points within the cluster depending on the characteristics of the data.



Visualization of K-Means



How does k-means clustering work?



How does k-means clustering work?

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in c_k} \|x_i - \mu_k\|^2$$

1. Specify number of clusters *K*.
2. Initialize centroids by randomly selecting K points.
3. Calculate the distance between each point in the dataset and each of the K centroids
4. Assign points to nearest centroid.

$$c_i = arg \min_k \|x_i - \mu_k\|^2$$

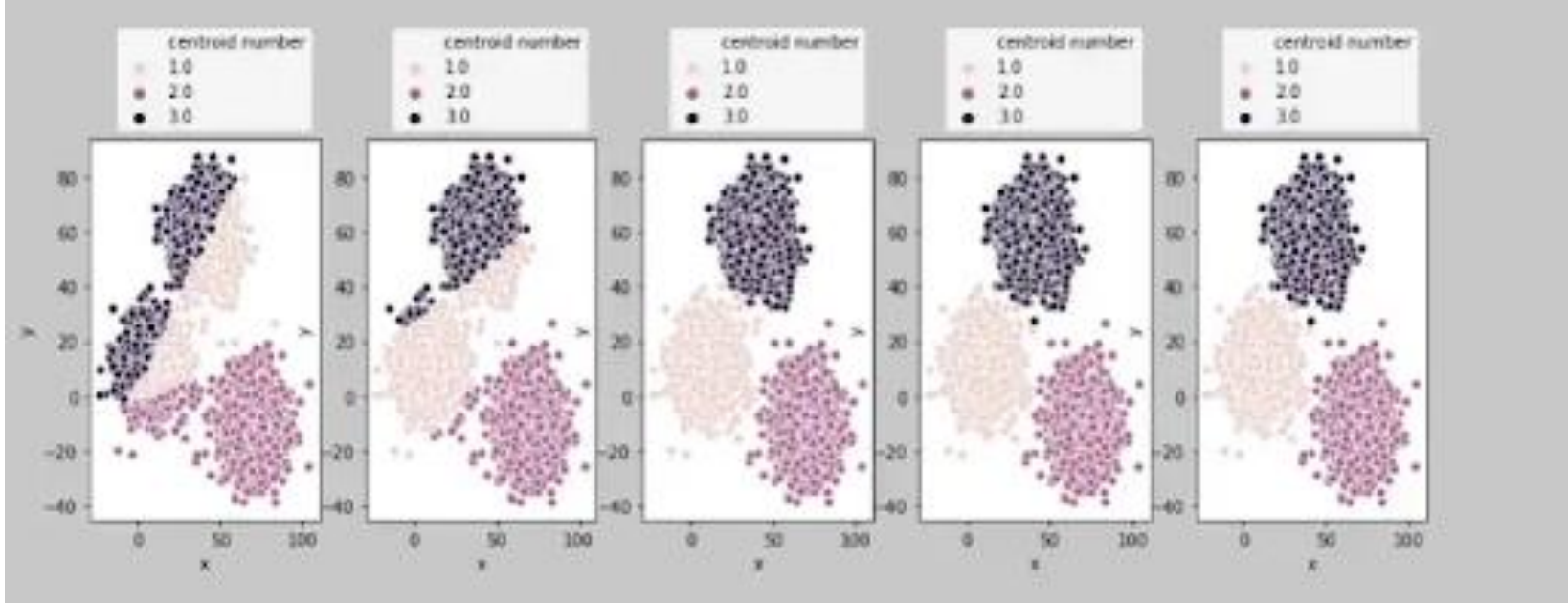
5. Update centroids: recalculate the centroids of the clusters by taking the mean of all data points assigned to each cluster.

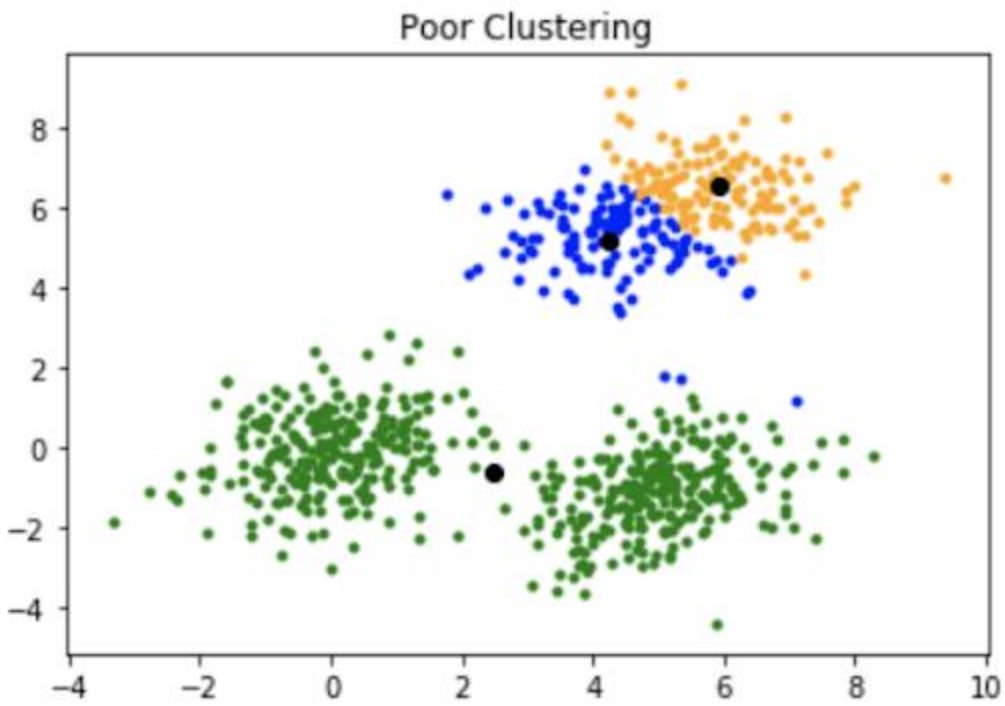
$$\mu_k = \frac{1}{|c_k|} \sum_{x_i \in c_k} x_i$$

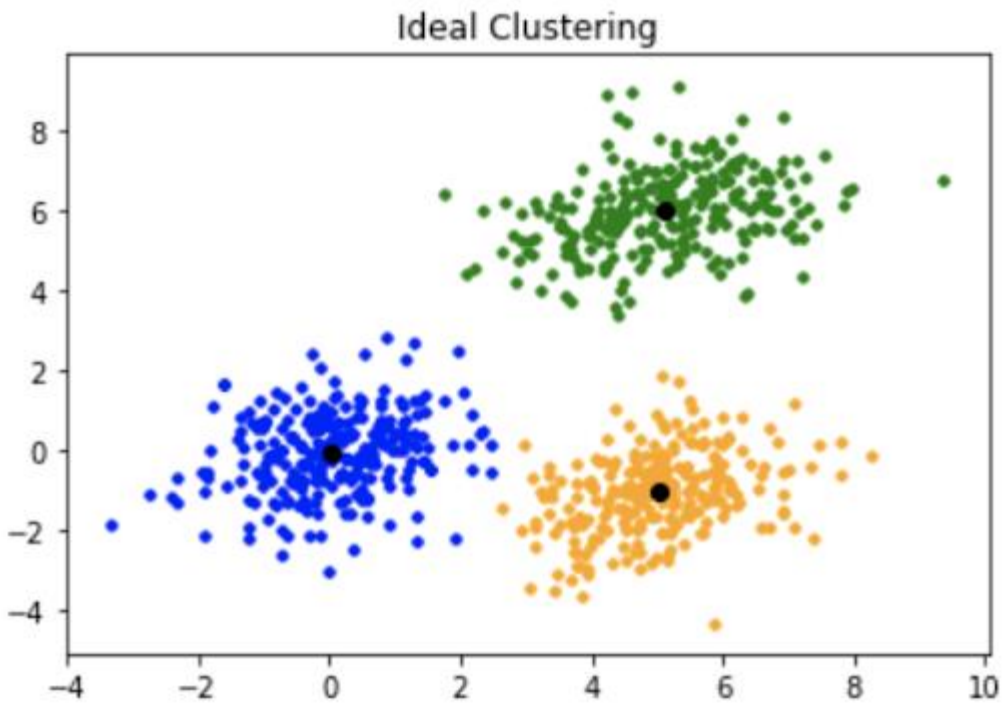
6. Repeat steps 2 and 3 until convergence.

Convergence

- Convergence occurs when the centroids no longer change significantly or when a specified number of iterations is reached (Stopping Criteria).




Poor Clustering
8
6
4
2
0
−2
−4
−4
−2
0
2
4
6
8
10


Ideal Clustering
8
6
4
2
0
−2
−4
−4
−2
0
2
4
6
8
10

Example

We have 6 data points (2D):

A(1,1) , B(1.5,2) , C(3,4) , D(5,7) , E(3.5,5) , F(4.5,5)

Step 1: Initialize centroids

- Centroid 1 (C1) = A(1,1)
- Centroid 2 (C2) = D(5,7)

Step 2: Assign points to nearest centroid

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Point	Distance to C1 (1,1)	Distance to C2 (5,7)	Cluster
A(1,1)	0	~7.21	C1
B(1.5,2)	~1.12	~6.10	C1
C(3,4)	~3.61	~3.61	(tie → choose C1)
D(5,7)	~7.21	0	C2
E(3.5,5)	~5.00	~2.50	C2
F(4.5,5)	~5.32	~2.24	C2

Step 3: Update centroids

Cluster 1 (A, B, C):

x mean = (1 + 1.5 + 3) / 3 = 1.83

y mean = (1 + 2 + 4) / 3 = 2.33

New C1 = (1.83, 2.33)

Cluster 2 (D, E, F):

x mean = (5 + 3.5 + 4.5) / 3 = 4.33

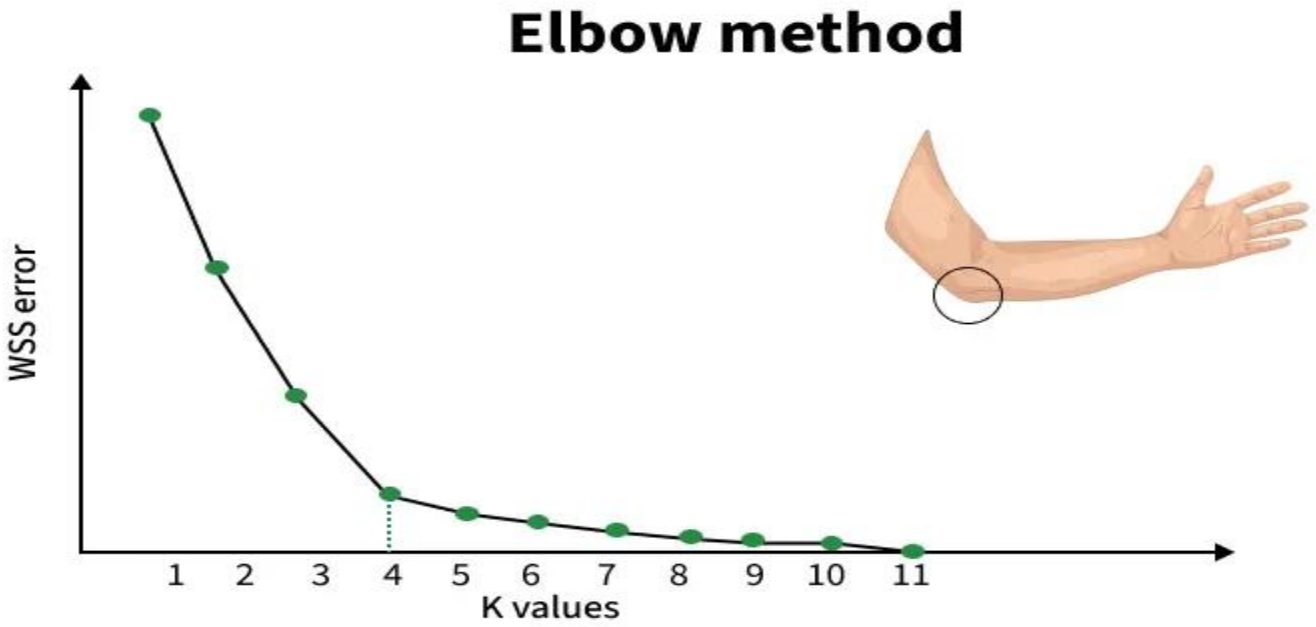
y mean = (7 + 5 + 5) / 3 = 5.67

New C2 = (4.33, 5.67)

Step 4: Repeat assignment

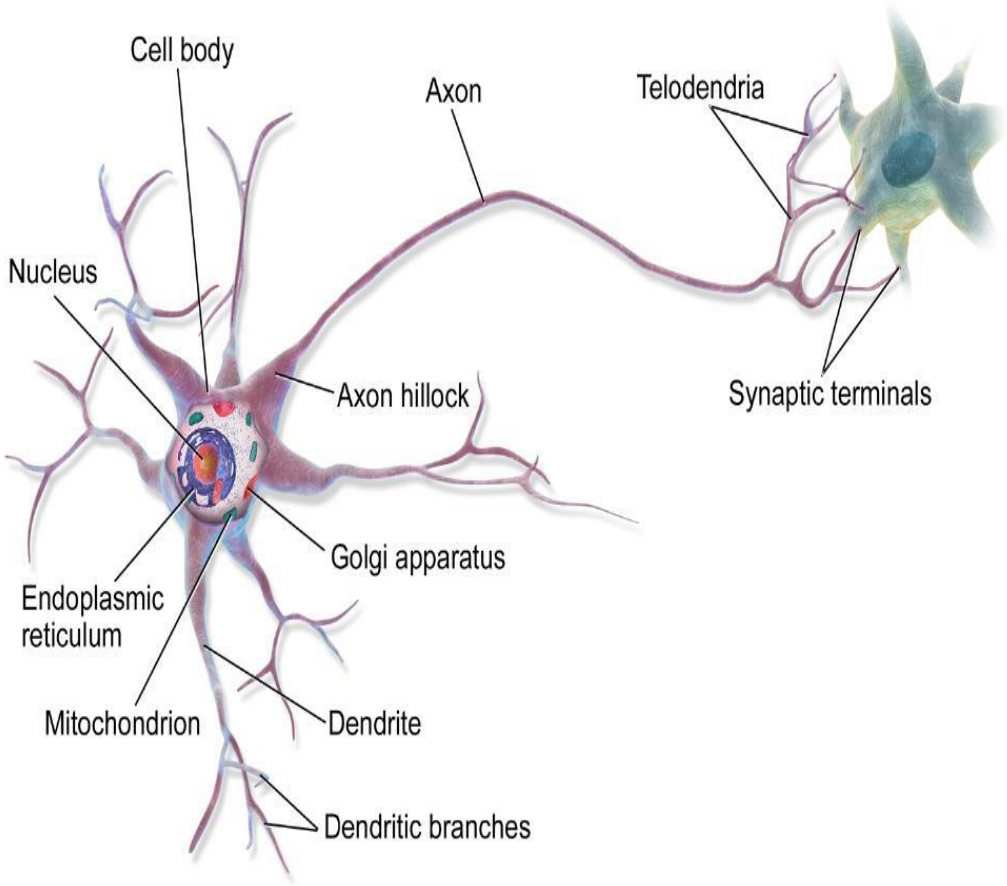
Elbow Method for optimal value of k in KMeans

- By plotting the Within-Cluster Sum of Squares (WCSS) against increasing k values and looking for a point where the improvement slows down.
- This point is called the "elbow."



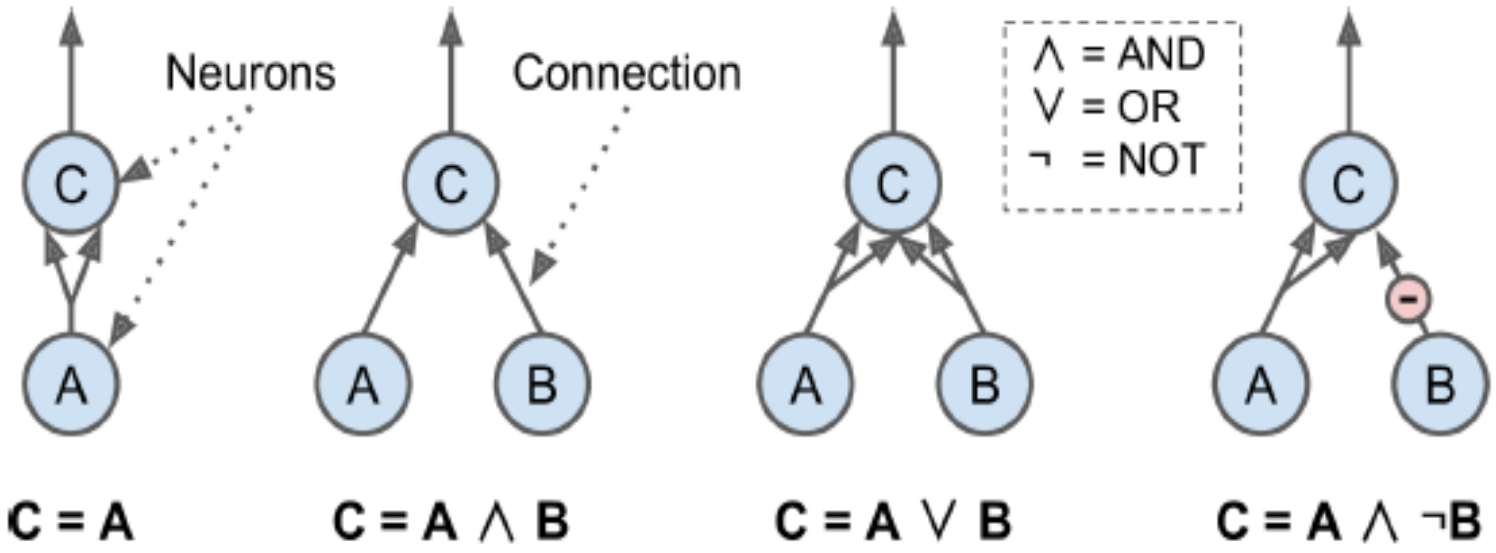
Introduction to Artificial Neural Networks

- Biological ***neurons*** receive short electrical impulses called ***signals*** from other neurons via *synapses*.
- When a neuron receives a sufficient number of signals within a few milliseconds, it ***fires*** its own signals.
- Organized in a vast network of billions of neurons.
- Each neuron typically connected to thousands of other neurons.



Logical Computations with Neurons

- Artificial neuron
  - It has one or more binary inputs and one binary output (a simplified model)
  - The output is activated when more than a certain number of its inputs are active
  - Artificial Neural Networks (ANN) can be built from many artificial neurons
- They can model any logical function

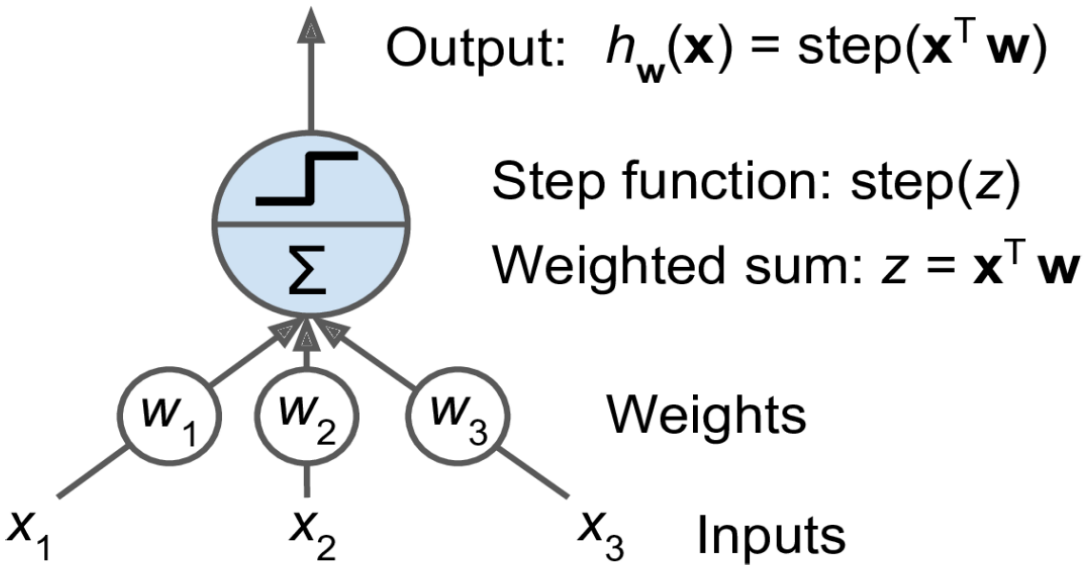


The Perceptron

- One of the simplest ANN architectures
  - Invented in 1957 by Frank Rosenblatt
- Based on a different artificial neuron called a threshold logic unit (TLU), or a linear threshold unit (LTU)

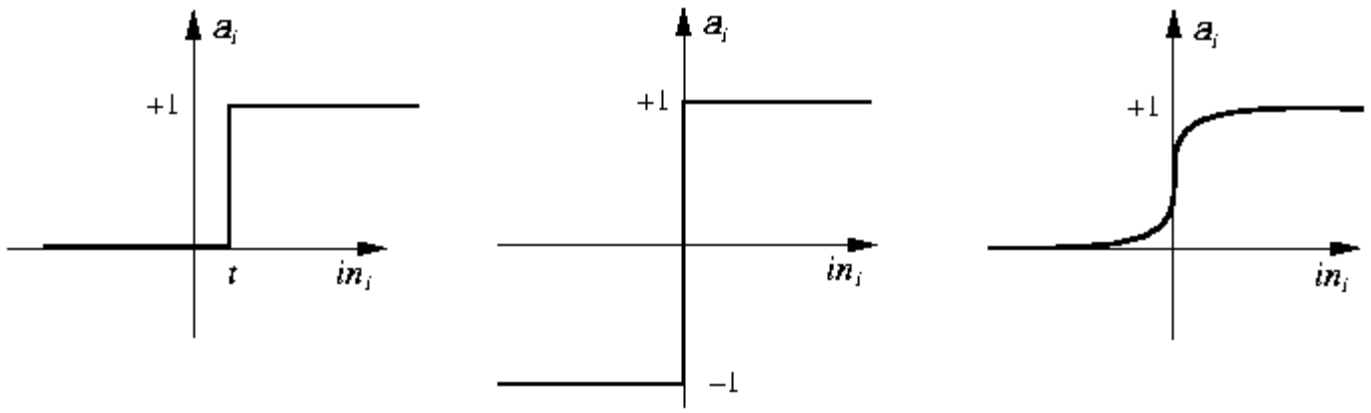
  - The inputs and output are numbers (with weights for the inputs)
  - Calculates a weighted sum of inputs then a step function
  - Common step functions:

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad \text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$



Common Activation Functions

- Stept(z)=1 if z >= t, else 0
- Sign(z)= +1 if z >= 0, else –1
- Sigmoid(z)=$\frac{1}{1+e^{-z}}$ -logistic function



(a) Step function (b) Sign function (c) Sigmoid function

Other Popular Activation Functions

- The hyperbolic tangent function

$$\tanh(z) == \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} = 2\sigma(2z) - 1$$

  – output value ranges from –1 to 1

  – make each layer's output more or less centered around 0 at the beginning of training

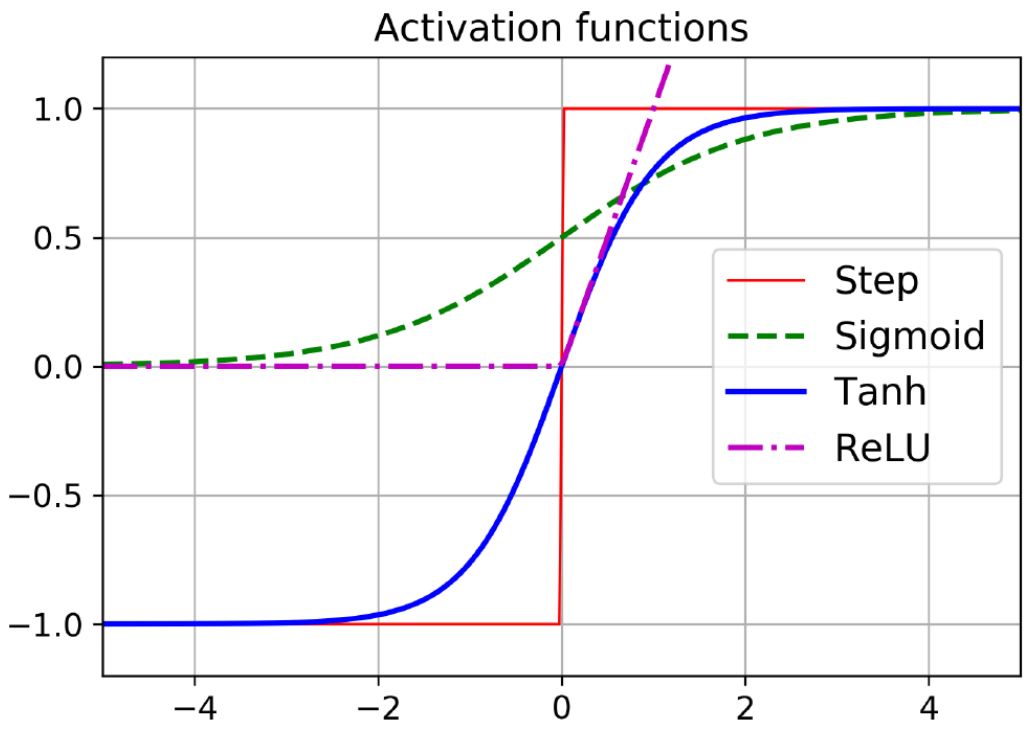
- Helps speed up convergence

Other Popular Activation Functions

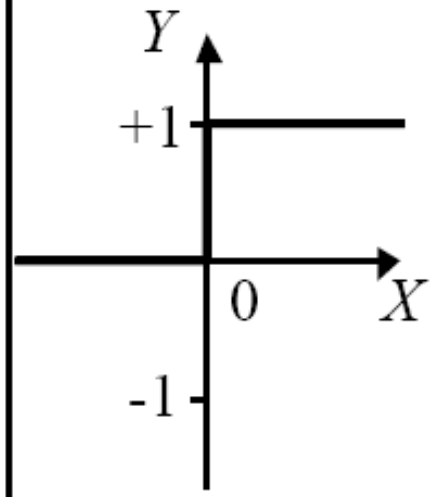
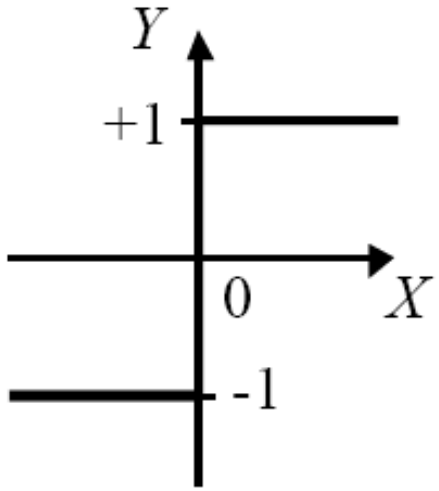
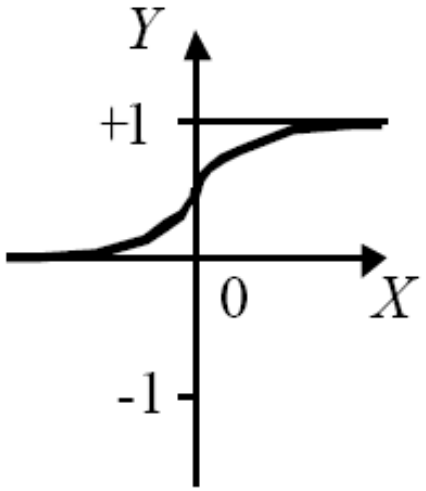
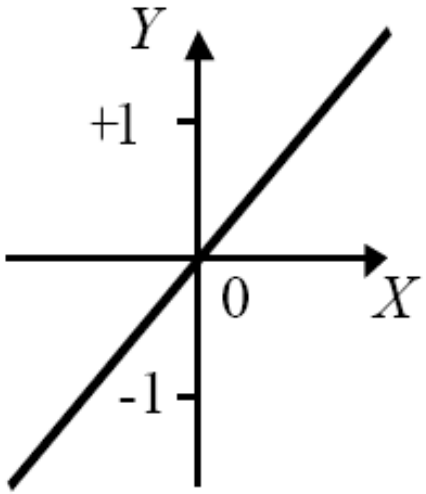
- The Rectified Linear Unit function:

$$\mathrm{ReLU}(z) = \max(0, z)$$

  – Fast to compute
  – It does not have a maximum output value

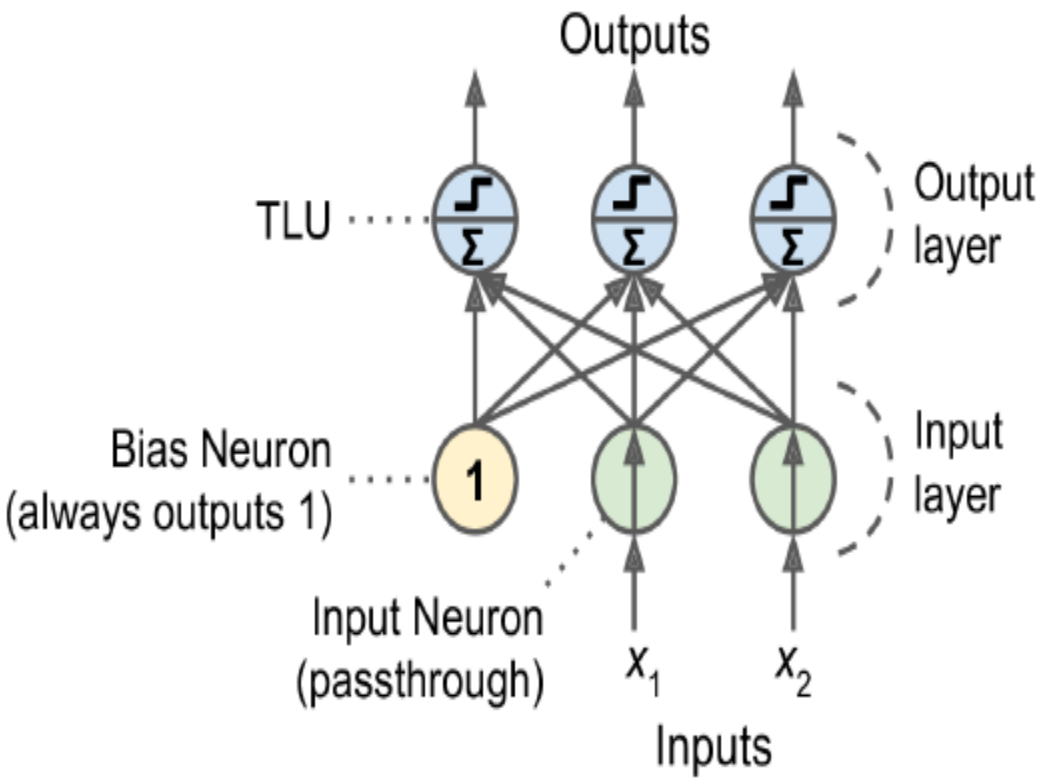


Activation Functions

Step function	*Sign function*	*Sigmoid function*	*Linear function*
hard limit functions			
			
$Y^{step} = \begin{cases} 1, \text{ if } X \geq 0 \\ 0, \text{ if } X < 0 \end{cases}$	$Y^{sign} = \begin{cases} +1, \text{if } X \geq 0 \\ -1, \text{if } X < 0 \end{cases}$	$Y^{sigmoid} = \dfrac{1}{1+e^{-X}}$	$Y^{linear} = X$

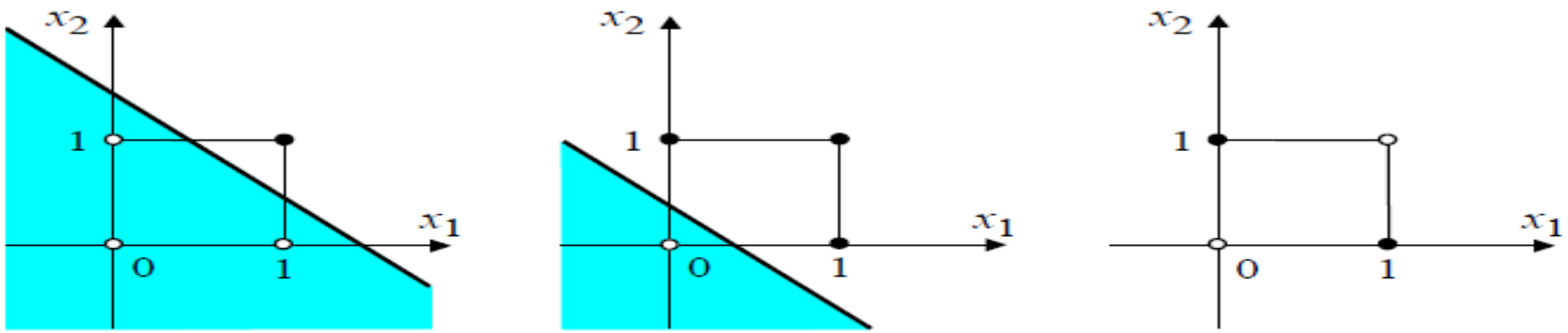
Perceptron diagram

- The output: $h_{\mathbf{W},\mathbf{b}}(\mathbf{X}) = \phi(\mathbf{XW} + \mathbf{b})$
  - X: matrix of input features
    - One row per instance and one column per feature
  - W: The weight matrix
    - One row per input neuron and one column per artificial neuron in the layer
  - B: Bias vector
    - All the connection weights between the bias neuron and the artificial neurons
  - ϕ: activation function.



Perceptrons and Linear Separability

- The decision boundary of Perceptron is linear
- Therefore, it is also known as a Linear Binary Classifier
- Some simple functions are not linearly separable
  - XOR function
  - Use Multi-Layer Neural Networks or Multi-Layer Perceptron (MLP)



(a) AND ($x_1 \cap x_2$) (b) OR ($x_1 \cup x_2$) (c) Exclusive-OR ($x_1 \oplus x_2$)

How does the perceptron learn?

- **Step 1: Initialization**
  - Set initial weights $w_1, w_2, \ldots, w_n$ and threshold to random numbers in the range [-0.5, 0.5]
- **Step 2: Activation**
  - Activate the perceptron by applying inputs $x_1(p)$, $x_2(p)$, $x_3(p)$, ...$x_n(p)$, and desired output $y_d(p)$ .Calculate the actual output at iteration $p = 1$

$$Y(p) = step\left[\sum_{i=1}^{n} x_i(p)w_i(p) - \theta\right]$$

  - where n is the number of the perceptron inputs, and step is a step activation function.

How does the perceptron learn?

- **Step 3: Weight training**
  - Update the weights of the perceptron

$$w_i(p+1) = w_i(p) + \Delta w_i(p),$$

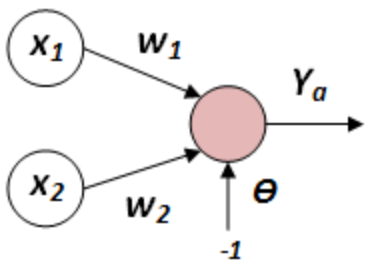
where Δw is the weight correction at iteration p. The weight correction is computed by the delta rule:

$$\Delta w_i(p) = \alpha \times x_i(p) \times e(p)$$

- **Step 4: Iteration**
  - Increase iteration p by one, go back to Step 2 and repeat the process until convergence.

Perceptron Example –AND

- Train a perceptron to recognize logical AND



Epoch	Inputs		Desired output	Initial weights		Actual output	Error	Final weights	
	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
1	0	0	0	0.3	–0.1	0	0	0.3	–0.1
	0	1	0	0.3	–0.1	0	0	0.3	–0.1
	1	0	0	0.3	–0.1	1	–1	0.2	–0.1
	1	1	1	0.2	–0.1	0	1	0.3	0.0
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	–1	0.2	0.0
	1	1	1	0.2	0.0	1	0	0.2	0.0

Epoch refers to one cycle through the full training dataset. Usually, training a **neural network** takes more than a few **epochs**

Use threshold $\Theta = 0.2$ and learning rate $\alpha = 0.1$

Perceptron Example –AND

Epoch 1

– Iteration 1 (input X_1=0; X_2=0; Y_d=0):

- (0 * 0.3 + 0 * -0.1) -0.2 = -0.2
- Y_a =step(-0.2) =0
- Error= Y_d - Y_a = 0
- Δw1=0; Δw2=0;

–Iteration 2 (input X_1=0; X_2=1; Y_d=0):

- (0 * 0.3 + 1 * -0.1) -0.2 = -0.3
- Y_a =step(-0.3) =0
- Error=Y_d- Y_a = 0
- Δw1=0; Δw2=0;

Perceptron Example –AND

–Iteration 3 (input X_1=1; X_2=0; Y_d=0)

- (1 * 0.3 + 0 * -0.1) -0.2 = +0.1
- Y_a =step(+0.1) =1
- Error=Y_d- Y_a = -1
- Δw_1= 0.1 * 1 * -1= -0.1
- w_1(new) =w_1(old)+ Δw_1 =0.3-0.1 =**0.2**
- $\boldsymbol{\Delta w_2}$ **=0**

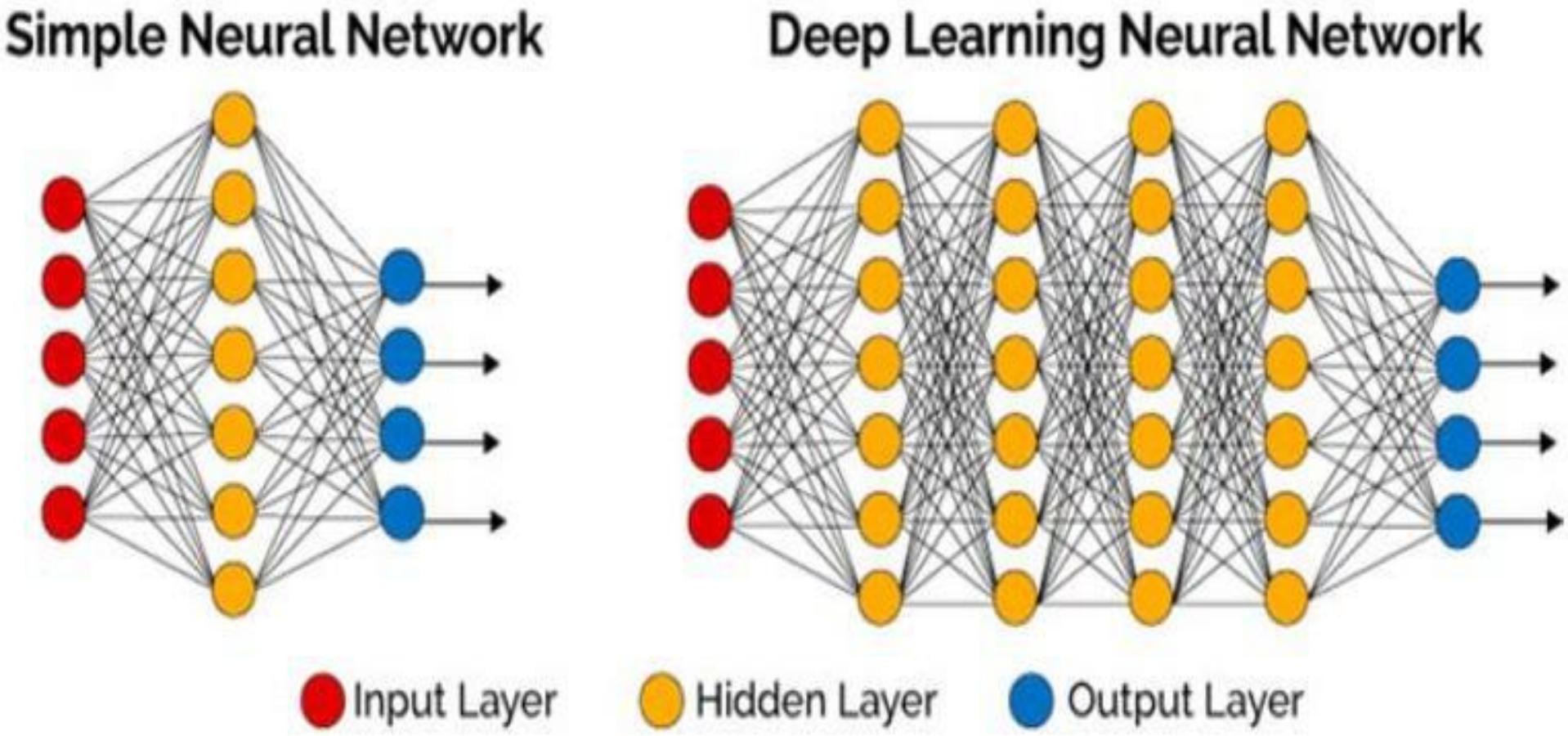
Perceptron Example –AND

Epoch	Inputs		Desired output	Initial weights		Actual output	Error	Final weights	
	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	–1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	–1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

- Repeat until convergence
  - i.e. final weights do not change and *no error*

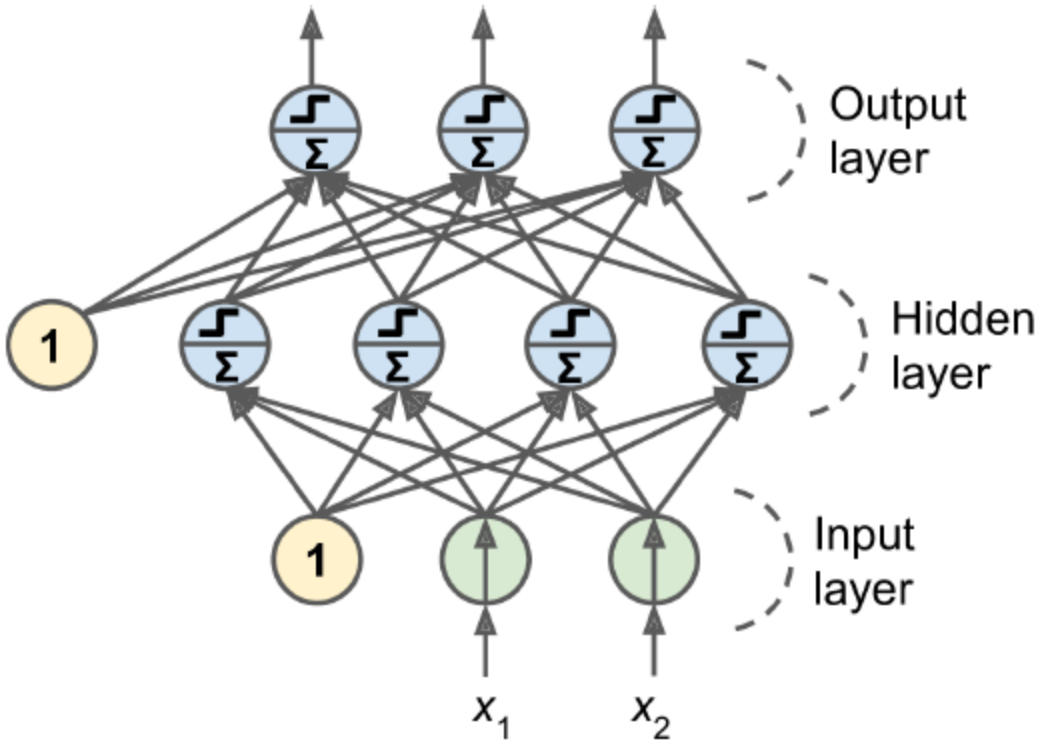
Epoch	Inputs		Desired output	Initial weights		Actual output	Error	Final weights	
	x_1	x_2	Y_d	w_1	w_2	Y	e	w_1	w_2
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

Feed-forward neural network



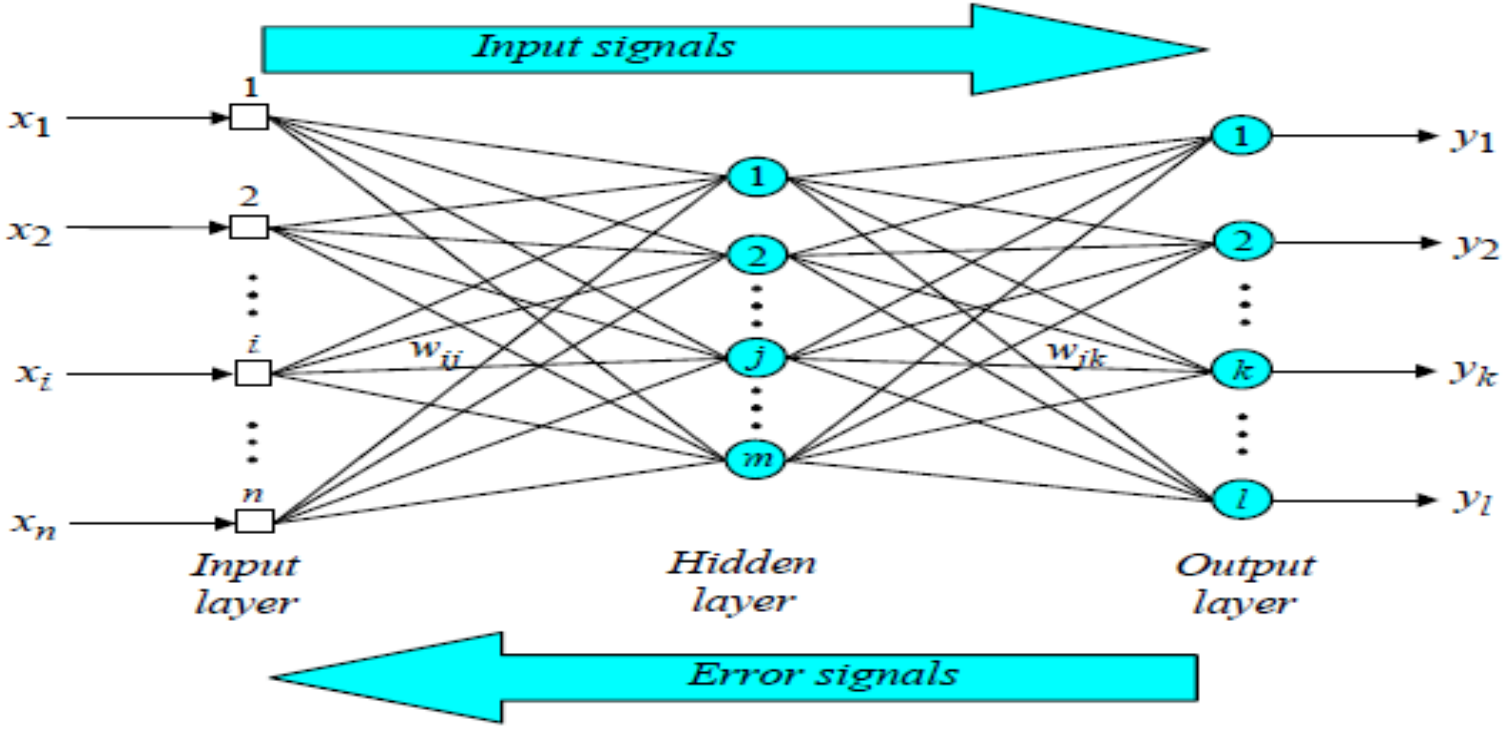
Multi-Layer Perceptron and Backpropagation

- Multi-Layer Perceptron (MLP)
  - One (pass through) input layer
  - One or more layers of TLUs -hidden layers
  - One final layer of TLUs called the output layer
- Every layer except the output layer includes a bias neuron and is fully connected to the next layer.
- When an ANN contains a deep stack of hidden layers, it is called a Deep Neural Network (DNN).



Backpropagation Training Algorithm

- Two passes through the network –forward and backward
  - – Compute the gradient of the network's error with regards to every single model parameter.
  - – Find out how each connection ***weight*** and each ***bias*** term should be ***adjusted*** in order to reduce the error.



Backpropagation Training Algorithm (Gradient Descent)

- Handles one mini-batch at a time (e.g. 32 instances)
- Each mini-batch is passed through all layers to calculate the output (prediction) – Forward pass
- Measures the network's output error
  - A loss function that compares the desired and the actual outputs
- Computes how much each output connection contributed to the error -by applying the chain rule
- Performs a Gradient Descent step to update all the connection weights in the network
- Goes through the full training set multiple times.
  - Each pass is called an epoch

Regression MLPs

- Use a single output neuron to predict a single value

  – One output neuron per output dimension for multivariate regression (i.e., to predict multiple values at once)

  - e.g. 1: locate the center of an object on an image (x, y)
  - e.g. 2: place a bounding box around the object (x, y, width, height)

- You do not need to use any activation function for the output neurons

  – They are free to output any range of values

  – For positive output → use the ReLU or the softplus activation functions in the output layer

  – Output within a given range → use the logistic function or the hyperbolic tangent →Scale the output (0 to 1, -1 to 1)

Regression MLPs

- Loss functions
  - Mean Squared Error (MSE)
  - Mean Absolute Error (MAE) -if you have a lot of outliers
  - Huber loss –combination of MSE and MAE
- Typical Regression MLP Architecture

Hyperparameter	Typical Value
# input neurons	One per input feature (e.g., 28 x 28 = 784 for MNIST)
# hidden layers	Depends on the problem. Typically 1 to 5.
# neurons per hidden layer	Depends on the problem. Typically 10 to 100.
# output neurons	1 per prediction dimension
Hidden activation	ReLU (or SELU, see Chapter 11)
Output activation	None or ReLU/Softplus (if positive outputs) or Logistic/Tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

❖ Mean Squared Error (MSE):

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Punishes large errors **a lot**
- Sensitive to outliers
- Units are squared (can feel unintuitive)
- Good when big mistakes are especially bad

❖ Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

❖ Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Treats all errors equally
- More **robust to outliers** than MSE

❖ R^2 (Coefficient of Determination)

$$R2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

- Proportion of variance
- Range:
  - 1 → perfect
  - 0 → same as predicting mean
  - < 0 → worse than mean

- **Relative Squared Error (RSE)**

$$RSE = \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

- **Relative Absolute Error (RAE)**

$$RAE = \frac{\sum_{i=1}^{n}|y_i - \hat{y}_i|}{\sum_{i=1}^{n}|y_i - \bar{y}_i|}$$

- **Mean Absolute Percentage Error (MAPE)**

$$MAPE = \frac{100}{2}\sum\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

  - **Breaks when y_i=0**

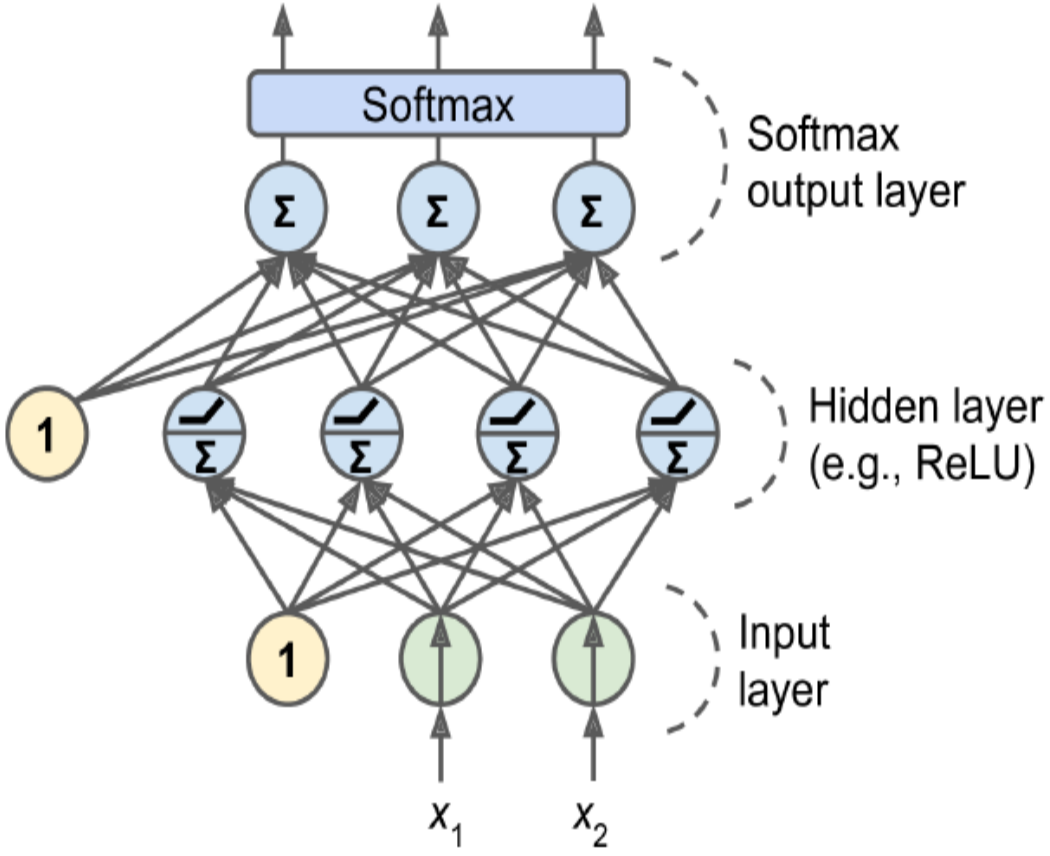
Classification MLPs

- Binary classification: Use a single output neuron
  - The estimated probability of the positive class
  - The estimated probability of the negative class: 1-the value above
- Multilabel binary classification
  - e.g. email classification into: ham or spam, and urgent or non-urgent → Two output neurons
  - Logistic activation function

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
Input and hidden layers	Same as regression	Same as regression	Same as regression
# output neurons	1	1 per label	1 per class
Output layer activation	Logistic	Logistic	Softmax
Loss function	Cross-Entropy	Cross-Entropy	Cross-Entropy

Classification MLPs

- Multiclass classification
  - 3 or more classes → one output neuron per class
  - e.g. classes 0 through 9 for digit image classification
  - Softmax activation function for the output
    - Estimates the probability of each class
      - Also called the normalized exponential)
    - All the estimated probabilities are between 0 and 1
  - Loss function: the cross-entropy
    - Also called the log loss



Softmax activation function

- For 2 classes, we use logistic regression

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This converts a score into a ***probability***.

- Now suppose we have ***K*** classes.

We want probabilities:

$$P(y=1), P(y=2), \ldots, P(y=K)$$

They must sum to 1

- Use Exponentials
  - Always positive → no negative probabilities
  - Bigger inputs grow **much faster** → clearer decisions
- So we define "unnormalized probabilities":

$$e^{z_1}, e^{z_2}, \ldots, e^{z_k}$$

- Normalize Them

$$P(y = i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Softmax takes a list of numbers (called **logits**) and turns them into **probabilities.**
  - All outputs are between 0 and
  - Sum to 1
  - Larger inputs → larger probabilities

Example

- A model predicts scores (logits) for 3 classes:
  - Cat → 1.2
  - Dog → 0.9
  - Bird → 0.4

$$z=[1.2, 0.9, 0.4]$$

- **Step 1: Apply exponentials**

$$e^{1.2} \approx 3.32, \;\; e^{0.9} \approx 2.46, e^{0.4} \approx 1.49$$

- **Step 2: Sum them**

$$3.32+2.46+1.49=7.27$$

- **Step 3: Normalize (divide by sum)**

$$P(\text{Cat}) = \frac{3.32}{7.27} \approx 0.457$$

$$P(\text{Dog}) = \frac{2.46}{7.27} \approx 0.338$$

$$P(\text{Bird}) = \frac{1.49}{7.27} \approx 0.205$$

https://playground.tensorflow.org/